1. Single Responsibility Principle (SRP)

Q1. Domain: E-Commerce

Problem Statement:
In an online shopping system, the `Product` class handles product information, price calculations (with discounts and taxes), stock management, and rendering product details to the UI.
Task:
Refactor the class to follow SRP by separating responsibilities into different classes or modules.

- Identify at least 3 different responsibilities.
- Provide updated class structure.

Q2. Domain: Healthcare

Problem Statement:
A `PatientRecord` class handles storing patient data, sending email notifications, and formatting records for printing.
Task:
Apply SRP by creating specialized classes for each responsibility.

- Show the refactored class diagram or code.
- Justify how each class has only one reason to change.

Q3. Domain: IoT Smart Home System

Problem Statement:
A `DeviceController` class manages device configuration, logs activity, communicates with external APIs, and handles user alerts.
Task:
Apply SRP to design a scalable architecture where each responsibility is separated.

- Refactor using best practices like composition or service delegation.
- Highlight benefits in terms of testability and scalability.

---

2. Open/Closed Principle (OCP)

Q4. Domain: Education / LMS

Problem Statement:
In a learning management system, the `ContentRenderer` class uses `if-else` or `switch` blocks to render different content types (video, article, quiz). Adding a new type (e.g., podcast) requires modifying this class.
Task:
Refactor the system so it's open for extension but closed for modification.

- Use polymorphism/interfaces to allow adding new content types easily.
- Show how new types plug in without altering the existing class.

Q5. Domain: E-Wallet App

Problem Statement:
An `OfferEngine` calculates discounts for different transaction types using a hardcoded structure. Adding new offer strategies breaks the engine.
Task:
Apply OCP to refactor the offer engine to support new discount strategies without modifying the engine itself.

- Implement using strategy pattern or dependency injection.
- Show how extensibility is achieved.

Q6. Domain: Insurance Claims Processing

Problem Statement:
A `ClaimProcessor` class handles life, vehicle, and health insurance claims using conditionals. The business wants to support additional claim types (travel, property) with minimal changes.
Task:
Redesign to follow OCP by introducing abstraction for claim processing.

- Show old vs. new class structure.
- Demonstrate how a new claim type is added without modifying existing code.

---

3. Liskov Substitution Principle (LSP)

Q7. Domain: Banking System

Problem Statement:
`Account` is a base class with methods `deposit()` and `withdraw()`. A derived class `FixedDepositAccount` throws exceptions if `withdraw()` is called.
Task:
Identify the LSP violation and refactor the design to comply with LSP.

- Use appropriate abstraction or interfaces.
- Provide an example showing correct substitution.

Q8. Domain: Vehicle Fleet Management

Problem Statement:
A `Vehicle` superclass has method `refuel()`. Subclasses like `ElectricCar` override it with empty logic or throw an exception.
Task:
Refactor the hierarchy to satisfy LSP.

- Introduce refined abstractions like `FuelVehicle` vs. `ElectricVehicle`.
- Demonstrate LSP through valid polymorphic behavior.

Q9. Domain: E-commerce Promotions

Problem Statement:
`Discount` is a base class with method `applyDiscount(cart)`. Subclasses like `NoDiscount`, `PercentageDiscount`, and `BOGO` override it. However, some override methods don't maintain the behavior expected by the clients using the base class.
Task:
Review and redesign to ensure all subclasses uphold LSP.

- Show a test case where the substitution fails and fix it.
- Explain how your fix preserves behavior consistency.

---

4. Interface Segregation Principle (ISP)

Q10. Domain: Food Delivery Platform

Problem Statement:
An interface `UserOperations` includes `placeOrder()`, `trackOrder()`, `rateDriver()`, and `manageRestaurant()`. But delivery users don't need restaurant management functionality.
Task:
Redesign the interface to comply with ISP.

- Split into role-specific interfaces (e.g., `CustomerOps`, `RestaurantOps`).
- Illustrate improved flexibility and reduced coupling.

Q11. Domain: Educational Software

Problem Statement:
An interface `ExamService` includes methods: `startExam()`, `gradeExam()`, `createQuestionBank()`, `assignTeacher()`. But automated proctoring systems only need a subset.
Task:
Break down the interface to smaller, client-specific ones following ISP.

- Explain how it benefits different clients like admin vs. student vs. AI systems.

Q12. Domain: Industrial IoT Monitoring

Problem Statement:
A unified interface `SensorDevice` includes `getReading()`, `resetCalibration()`, `uploadFirmware()`, `startSelfTest()`. However, simple passive sensors only support `getReading()` and throw errors for others.
Task:
Refactor the interfaces using ISP.

- Create minimal interfaces for different device types.
- Demonstrate how modular sensor integration becomes easier.

---

5. Dependency Inversion Principle (DIP)

Q13. Domain: Fintech Loan Platform

Problem Statement:
The `LoanProcessor` class directly depends on `CreditScoreAPI` and `DocumentVerifier`. Any change in third-party APIs breaks the processor.
Task:
Apply DIP to decouple `LoanProcessor` from low-level modules.

- Introduce interfaces and inject dependencies.
- Explain how this improves maintainability and testability.

Q14. Domain: E-Commerce Notifications

Problem Statement:
An `OrderService` directly calls `EmailSender`, `SMSSender`, and `PushNotifier`. If any service fails or changes, the order service is affected.
Task:
Use DIP to abstract notification logic.

- Implement a notifier interface and use dependency injection.
- Add a new notification type (e.g., WhatsApp) without touching `OrderService`.

Q15. Domain: Cloud DevOps Monitoring System

Problem Statement:
A monitoring dashboard collects logs from AWS, Azure, and GCP using hard-coded logic. Each vendor API is tightly integrated into the `DashboardService`.
Task:
Refactor the system using DIP.

- Abstract log retrieval through an interface.
- Add a mock log provider for testing without modifying core code.